INGEGNERIA DEL SOFTWARE

ESERCITAZIONE 4

Gabriele Paveri-Fontana
Mail: gabriele.paveri@polimi.it

28 Ottobre 2026

Esercizio 1 Vacation Resort (1/2)

Describe through a UML Class Diagram a management system for the buildings of a vacation resort.

There are different kinds of buildings: apartments, small villas and big villas. Each building has an id and is characterized by a size and a maximum number of occupants. Each building can have a number of optional features: washing machine, dishwasher, garden, parking place, sea overlook.

The buildings can be reserved separately (i.e., each reservation concerns a single building) and for any number of days. Each reservation is made by a person (the contact-person for the reservation), for a group of people (the occupants of the building, including the contact person). The system keeps track, for each occupant, of his/her personal data (first name, last name, birth date, nationality, number of id card).

Esercizio 1 Vacation Resort (2/2)

The rental price of each building varies during the season.
In some cases the rental price can be discounted.

The system allows users to check whether, given the id of a building and a target reservation period, the building is free in that period or not; the system also allows users to book a building in a target period, by providing in the reservation also the occupants.

Finally, the system offers functions that, given a building and a target period, calculate the total cost of renting the building for that period, and what discount can be applied.

Recap

Design Pattern

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	Factory Method	Adapter	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter Bridge Composite Decorator Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Flyweight Observer State Strategy Visitor

Esercizio 2 Formatting Strategy

Implementare un'applicazione *TextEditor* di elaborazione del testo che permette di eseguire diverse procedure di formattazione.

Specifiche:

- Il *TextEditor* ha un attributo di tipo *String* (il testo da formattare) ed un metodo *performFormatting()* per eseguire le procedure di formattazione.
- Le procedure di formattazione che deve essere possibile eseguire sono:
  - ✓ *UpperCase*: converte il testo in maiuscolo;
  - ✓ *LowerCase*: converte il testo in minuscolo;
  - ✓ *TitleCase*: converte il testo in modo che l'inizio di ogni parola sia in maiuscolo e il resto in minuscolo;
  - ✓ Ecc…

Esercizio 2 Formatting Strategy

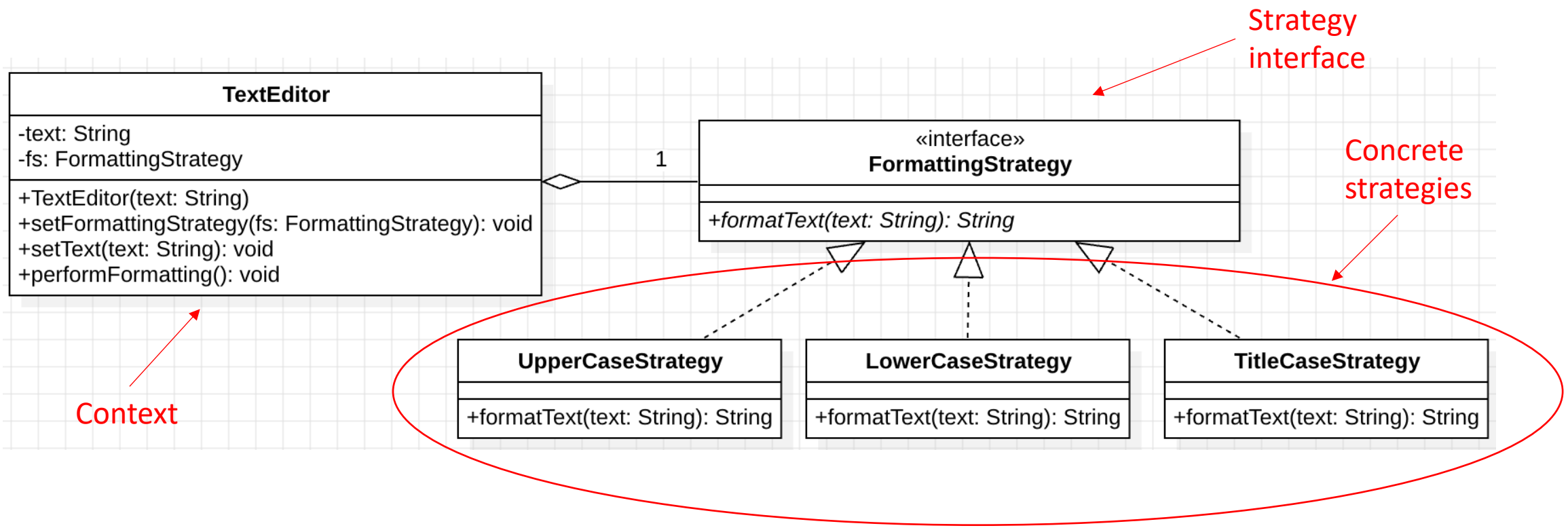
Antipattern:

```
public class TextEditor {
    private String text;
    public TextEditor(String text) {
        this.text = text;
    }
    public performFormatting (int type) {
        if (type == 1)
      //implementazione di UpperCase
        if (type == 2)
      //implementazione di LowerCase
        …
    }
}
```

Esercizio 2 Formatting Strategy

Pattern: ***Strategy*** (Behavioral Pattern)

- Per definire una famiglia di algoritmi intercambiabili dinamicamente.



Esercizio 3 Drink Decorator

Implementare un'applicazione che gestisce un semplice bar, dove è possibile ordinare una bevanda con l'aggiunta di vari condimenti.

Specifiche:

- Una bevanda (caffè, tè, ecc…) ha un costo base.
- I condimenti che si devono poter aggiungere ad una bevanda sono:
  - ✓ Zucchero, al costo aggiuntivo di 20 cent;
  - ✓ Latte, al costo aggiuntivo di 50 cent;
  - ✓ Ecc…
- Si devono poter aggiungere più condimenti alla stessa bevanda.

Esercizio 3 Drink Decorator

Antipattern:

```
public class PlainTea {
    public double getCost() {
            return 2.0;
    }
}

public class SugarTea extends
PlainCoffee {
    public double getCost() {
            return 2.2;
    }
}

…
```

```
public class PlainCoffee {
    public double getCost() {
            return 1.0;
    }
}

public class SugarCoffee extends PlainCoffee {
    public double getCost() {
            return 1.2;
    }
}

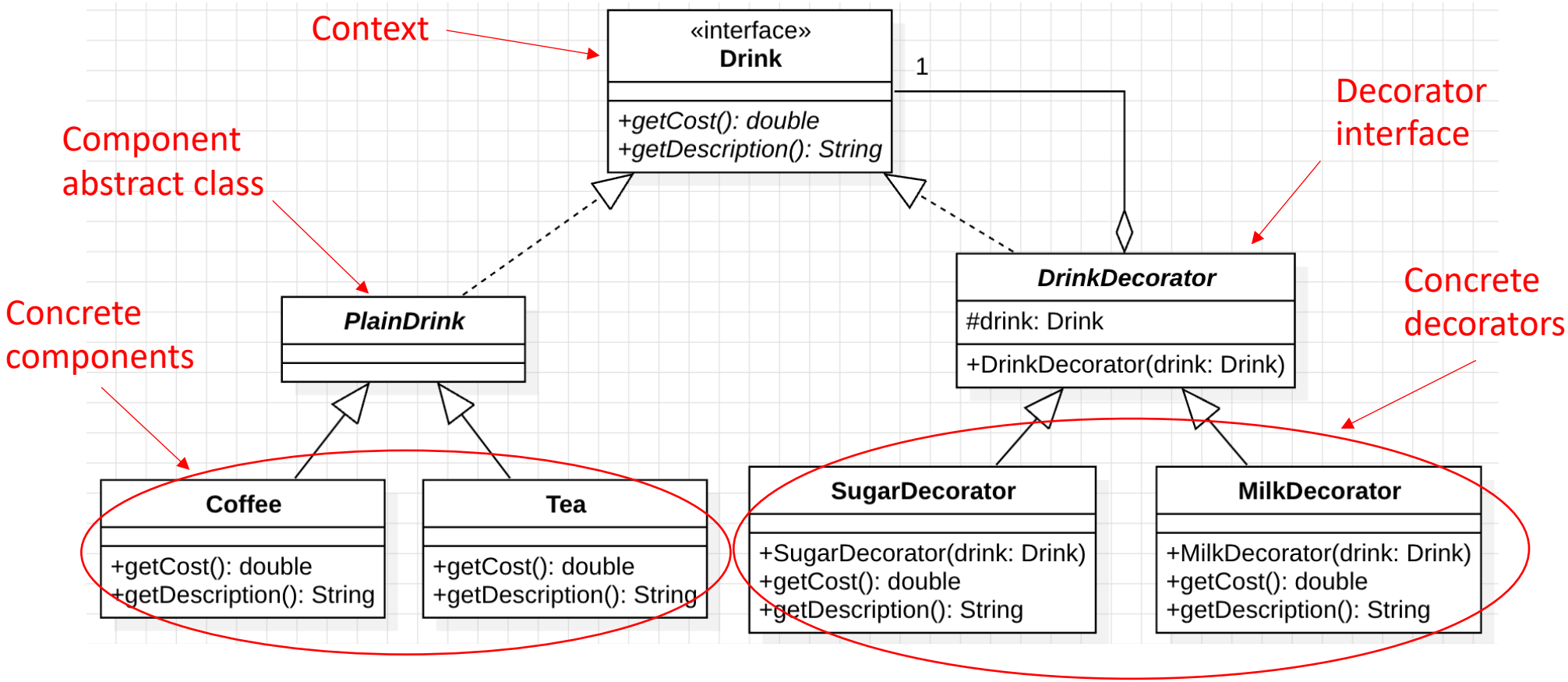
public class MilkCoffee extends PlainCoffee {
    public double getCost() {
            return 1.5;
    }
}

public class Sugar&MilkCoffee extends PlainCoffee {
    public double getCost() {
            return 1.7;
    }
}
```

Esercizio 3 Drink Decorator

Pattern: *Decorator* (Structural Pattern)

- Per poter aggiungere ad un oggetto nuove funzionalità (multiple) a runtime.



Esercizio 4 Pet Factory

Implementare un'applicazione che gestisce la creazione di vari tipi di animali domestici e permette ad un cliente di procurarsene uno.

Specifiche:

- Un *Client* ha un attributo di tipo *Pet* ed un costruttore che gli assegna una nuova istanza del *Pet* desiderato.
- I *Pet* che deve essere possibile creare sono:
  - ✓ Cani;
  - ✓ Gatti;
  - ✓ Ecc…

Esercizio 4 Pet Factory

Antipattern:

```
public class Client {

    private Pet pet;

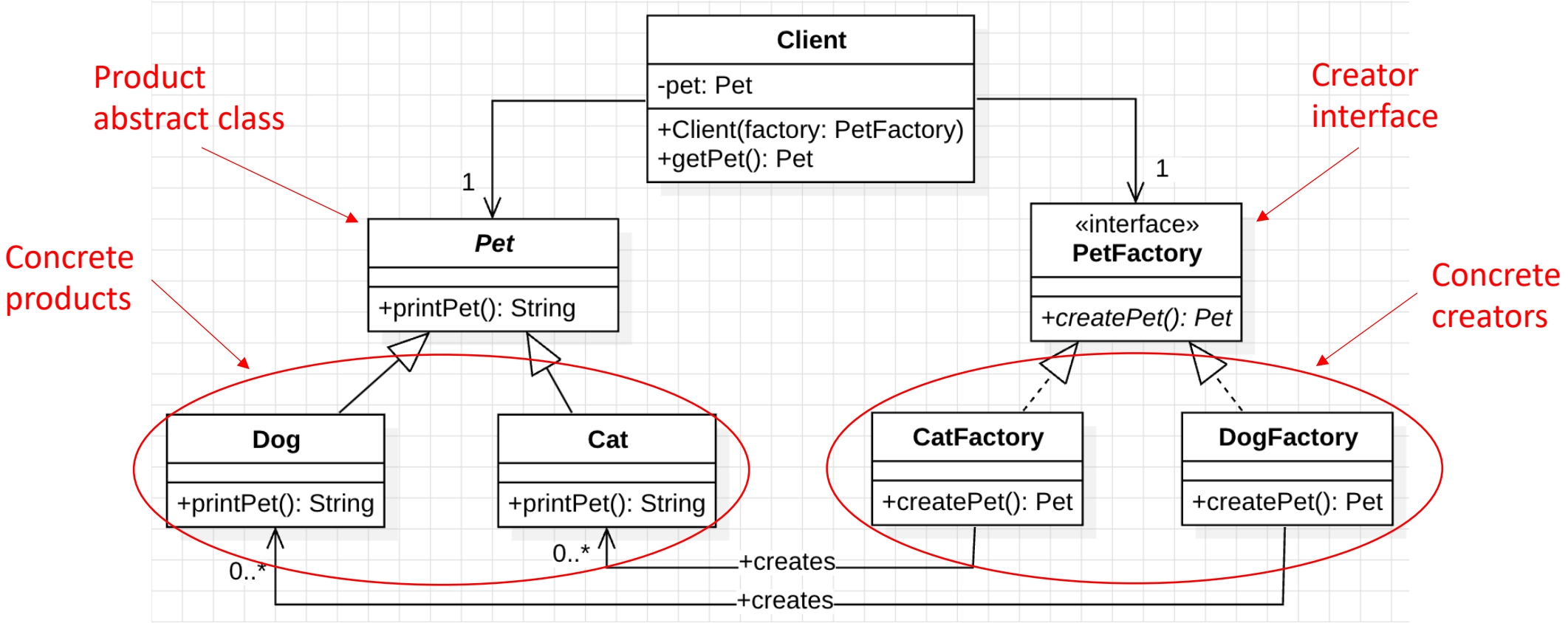
    public Client (String petType) {
        if (petType == “Dog”)
            pet = new Dog();
        if (petType == “Cat”)
            pet = new Cat();
        …
    }

}
```

Esercizio 4 Pet Factory

Pattern: *Abstract factory* (Creational Pattern)

- Per consentire ad una classe di differire la creazione di oggetti alle sue sottoclassi, delegando la responsabilità ad un metodo di una classe helper.



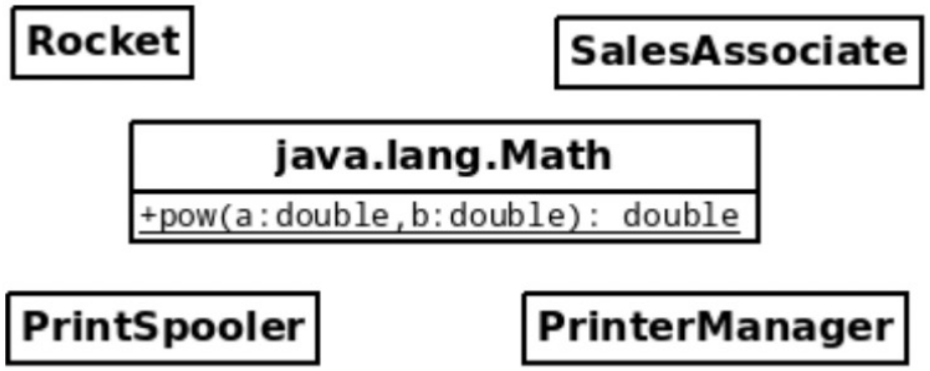
Esercizio 5 Singleton

Pattern: ***Singleton*** (Creational Pattern)

- Per garantire che una classe abbia una e una sola istanza (il pattern fornisce anche un punto di accesso globale all'unica istanza).

Domande:

1) Tra le classi dell'esercizio 1, quali potrebbero applicare il pattern singleton?
2) E tra le seguenti classi?



Esercizio 5 Singleton

Pattern: *Singleton* (Creational Pattern)

- Per garantire che una classe abbia una e una sola istanza (il pattern fornisce anche un punto di accesso globale all'unica istanza).

Risposte:

1) La classe *System* e nessun'altra.
2) *Rocket* e *SalesAssociate*: non possono essere Singleton, è probabile che siano normali e comuni classi (sono tipi di impiegati di un ufficio).
   java.lang.Math: ha metodi statici, dunque non può essere un Singleton.
   PrintSpooler: lo spooler di stampante è in ogni stampante, dunque non può essere un Singleton.
   PrinterManager: un gestore di tutte le stampanti è decisamente un singleton (una sola istanza per gestire tutte le stampanti di un ufficio).

Esercizio 6 Internet of things (1/2)

Describe through a UML Class Diagram part of an “Internet of Things” (IoT) system that collects data from various kinds of sensors.

Each sensor reads a quantity, and provides it to a collector. Each sensor has an identifier, and a position (which is given in terms of its GPS coordinates).
There are 2 main types of sensors: those that provide data in a “push” fashion (i.e., they send the collected data to the collector), and those that provide it in a “pull” fashion (i.e., they wait to be invoked to provide the data).

Sensors that provide data in a “push” fashion do so with a certain frequency (expressed in Hertz), which is a configurable feature of the sensor.
Sensors that provide data in a “pull” fashion, instead, are characterized by the frequency (again, in Hertz) with which they refresh the data read from the field, and which is also configurable.

Esercizio 6 Internet of things (2/2)

In addition, there are other, orthogonal, categories of sensors, depending on their size: normal, small, and tiny. Sensors that are tiny and that provide their data in a “push” fashion can be put to sleep, and then can be re-awakened by the data collector.

Each sensor, independent of its nature, has a battery, which is characterized by its state of charge (which is expressed as the percentage to being fully charged).

Finally, the system must also feature a notification management system that notifies the user whenever a sensor, regardless of its type, changes its state. Can you think of any design pattern(s) to model this last feature?